

Self-Evolving AI: A Unified Framework for Generation, Evaluation, Memory, and Improvement

A Survey of Self-Improvement Loops in Large Language Models,
Evolutionary Computation, and Autonomous Agents

Self Evolve Research Team

<https://github.com/copizzah/awesome-evolution>

May 2026

Abstract

AI self-evolution is not a single technique but a convergence of five fundamental loops: specification-to-execution, search, evaluator, reflection, and population. This survey systematically reviews the emerging landscape of self-improving AI systems, spanning 19+ open-source projects and 17+ foundational papers across five research communities—AutoML, Neural Architecture Search (NAS), LLM self-improvement, evolutionary computation, and autonomous agent frameworks. We propose a unified taxonomy that classifies existing work along the five evolution loops, revealing that progress in each community independently rediscovers mechanisms that others have formalized. We analyze self-improvement methods at both inference time (Self-Refine, Reflexion, Self-Debug) and training time (SPIN, STaR, Constitutional AI), trace the emergence of LLM-driven evolutionary optimization from OPRO to AlphaEvolve, and map the frontier of agent self-evolution from fixed architectures to open-ended self-modifying systems (Darwin Gödel Machine, Gödel Agent, Absolute Zero). We identify critical gaps in current agent frameworks—the absence of persistent evaluation, lineage tracking, and cross-run memory—and position Self-Evolve as the missing evolution layer above existing orchestration frameworks. Our analysis covers the researcher ecosystem, open problems in evaluation reliability, memory stability, safety alignment, and composability, and provides a roadmap from research to production self-evolving AI systems.

Keywords: Self-Evolution, LLM Self-Improvement, Evolutionary Computation, Agent Evolution, AutoML, Neural Architecture Search, Autonomous Agents

Contents

1	Introduction	4
1.1	From Static AI to Self-Evolving AI	4
1.2	What is AI Self-Evolution?	4
1.3	Relation to Existing Concepts	4
1.4	Contributions	5
1.5	Paper Structure	5
1.6	Related Surveys	5
2	Taxonomy: Five Evolution Loops	5
2.1	Specification-to-Execution Loop	6
2.2	Search Loop	6
2.3	Evaluator Loop	7
2.4	Reflection Loop	7
2.5	Population Loop	8
2.6	Cross-Loop Comparison and Interactions	8

3	Self-Improvement Methods	9
3.1	Inference-Time Self-Improvement	9
3.1.1	Self-Refine	9
3.1.2	Reflexion	9
3.1.3	Self-Debug	10
3.2	Training-Time Self-Improvement	10
3.2.1	SPIN: Self-Play Fine-Tuning	10
3.2.2	STaR: Self-Taught Reasoner	10
3.2.3	Constitutional AI	10
3.2.4	ReST: Reinforced Self-Training	10
3.3	Inference-Time vs. Training-Time Trade-offs	11
3.4	Comparative Analysis	11
4	Evolutionary Code and Algorithm Discovery	11
4.1	LLM as Optimizer	11
4.1.1	OPRO: Optimization by PROMPTing	11
4.2	LLM as Evolutionary Operator	12
4.2.1	AlphaEvolve	12
4.2.2	LLaMEA: Evolving the Optimizer	12
4.3	Program Evolution and Mathematical Discovery	12
4.3.1	FunSearch	12
4.4	Open-Source Implementations	13
4.5	Challenges and Limitations	13
4.6	Broader Impacts	13
5	Agent Self-Evolution Systems	14
5.1	Agent Architecture Self-Evolution	14
5.1.1	ADAS: Automated Design of Agentic Systems	14
5.1.2	Gödel Agent	14
5.1.3	Darwin Gödel Machine	15
5.2	Agent Workflow Self-Evolution	15
5.3	Agent Memory and Learning	15
5.3.1	Agent Symbolic Learning	15
5.3.2	Absolute Zero	16
5.4	Multi-Agent Collaborative Evolution	16
5.5	Toward a Hierarchy of Self-Evolution	16
6	Agent Frameworks and Infrastructure	16
6.1	Framework Overview and Comparison	16
6.2	Evolution Capability Gaps	17
6.3	Self-Evolve Positioning: Evolution Layer Above Frameworks	17
7	AutoML and NAS Meets LLM	17
7.1	AutoML + LLM	18
7.2	NAS + LLM	18
7.3	The Text-to-ML Paradigm	18
7.4	Architecture as Genome: The NAS Metaphor	18
7.5	Unified Perspective with Self-Evolution	18
8	Research Landscape and Community	19
8.1	Core Research Groups	19
8.2	Academic Lineage	19
8.3	Cross-Community Convergence	19
8.4	Open-Source Ecosystem	19

9	Open Problems and Future Directions	19
9.1	The Evaluation Bottleneck	20
9.2	Memory Stability and Drift	20
9.3	Safety and Alignment in Self-Modifying Systems	20
9.4	Composability of Evolution Loops	21
9.5	From Research to Production	21
9.6	The Self-Evolve Roadmap	22
10	Conclusion	22
A	Benchmark Data Summary	23
B	Core Papers Analyzed	23

1 Introduction

The history of artificial intelligence is characterized by alternating waves of static, hand-designed systems and adaptive, self-improving ones. Early expert systems required manual knowledge engineering; statistical machine learning introduced automated parameter fitting; deep learning automated feature engineering; and now, large language models (LLMs) are beginning to automate their own improvement. We stand at the threshold of a new paradigm: **self-evolving AI**—systems that can generate, evaluate, remember, and improve upon their own behavior without explicit human intervention at each step.

1.1 From Static AI to Self-Evolving AI

The trajectory from static to self-evolving AI can be understood through a series of expanding automation scopes:

1. **Static systems** (pre-2012): Hand-crafted rules, features, and architectures. Every component designed by humans.
2. **Learned representations** (2012–2017): Deep learning automated feature engineering, but architectures and training procedures remained hand-designed.
3. **Architecture search** (2017–2020): Neural Architecture Search (NAS) and AutoML began automating the design of model architectures and ML pipelines.
4. **Prompt-driven systems** (2020–2023): LLMs enabled natural language specification of tasks, with systems like AutoML-GPT converting task descriptions into complete ML pipelines.
5. **Self-evolving systems** (2023–present): Systems that can modify their own code, prompts, workflows, and architectures through iterative cycles of generation, evaluation, and improvement.

Each transition automated what was previously considered an inherently human creative act. Self-evolving AI represents the current frontier: systems that improve their own improvement process.

1.2 What is AI Self-Evolution?

We define **AI self-evolution** as the capacity of an AI system to autonomously improve its performance through iterative cycles that involve:

- **Generation:** Producing candidate solutions, architectures, prompts, code, or agent behaviors.
- **Evaluation:** Scoring candidates against task objectives, benchmarks, or deployed metrics.
- **Memory:** Storing successful and failed attempts for future reference.
- **Improvement:** Using accumulated experience to produce better candidates in subsequent iterations.

This definition deliberately spans multiple research communities. A prompt optimization system (OPRO) and an agent architecture search system (ADAS) both exhibit self-evolution, even though they operate on different substrates (text prompts vs. agent code) and use different mechanisms (LLM-as-optimizer vs. foundation model-guided search). The common thread is the presence of an iterative improvement cycle where the system’s own outputs serve as inputs to its next improvement step.

1.3 Relation to Existing Concepts

AI self-evolution intersects with but is distinct from several established concepts:

- **Continual learning** focuses on learning from sequential data streams without catastrophic forgetting. Self-evolution focuses on improving the *system itself* (its prompts, code, architecture), not just updating a model on new data.

- **Online learning** updates model parameters from streaming data. Self-evolution may involve parameter updates but also encompasses non-parametric improvements (prompt refinement, code mutation, workflow restructuring).
- **Self-supervised learning** creates training signals from the data itself. Self-evolution creates improvement signals from the system’s own execution trace and evaluation feedback.
- **Meta-learning** learns how to learn. Self-evolution is a form of meta-learning but extends beyond parameter adaptation to include structural and behavioral modification of the agent itself.

1.4 Contributions

This survey makes the following contributions:

1. A **unified taxonomy of five evolution loops** that classifies self-evolution mechanisms across five research communities (Section 2).
2. A systematic analysis of **19+ open-source projects** with detailed technical comparisons (Sections 4–6).
3. A **cross-domain perspective** that unifies AutoML, NAS, LLM self-improvement, evolutionary computation, and agent frameworks under a single framework.
4. An analysis of the **researcher ecosystem** including institutional relationships and citation networks (Section 8).
5. Identification of **open problems** and a roadmap for building production self-evolving systems (Section 9).

1.5 Paper Structure

The remainder of this paper is organized as follows. Section 2 presents our five-loop taxonomy. Section 3 reviews self-improvement methods at inference and training time. Section 4 covers evolutionary code optimization and algorithm discovery. Section 5 surveys agent self-evolution systems. Section 6 reviews agent frameworks and their evolution capability gaps. Section 7 examines the intersection of AutoML, NAS, and LLMs. Section 8 maps the research community. Section 9 discusses open problems and future directions. Section 10 concludes.

1.6 Related Surveys

Several surveys have examined subsets of the self-evolution landscape. The survey on self-evolution of LLMs [?] organizes the LLM self-improvement process into four stages: experience acquisition, experience optimization, model update, and evaluation. While comprehensive for LLM-specific methods, this framework does not encompass the broader landscape of evolutionary computation, agent self-modification, and AutoML/NAS that we cover. RISE [?] examines recursive self-improvement but focuses on the theoretical foundations rather than practical systems. Our survey differs by providing a unified taxonomy that spans all five research communities, analyzing 19+ open-source implementations, and identifying concrete gaps in production infrastructure.

2 Taxonomy: Five Evolution Loops

We propose that self-evolving AI systems can be understood through the lens of five fundamental evolution loops. Each loop represents a distinct pattern of self-improvement that has been independently discovered and refined by different research communities. Crucially, these loops are not mutually exclusive—the most powerful self-evolving systems combine multiple loops simultaneously.

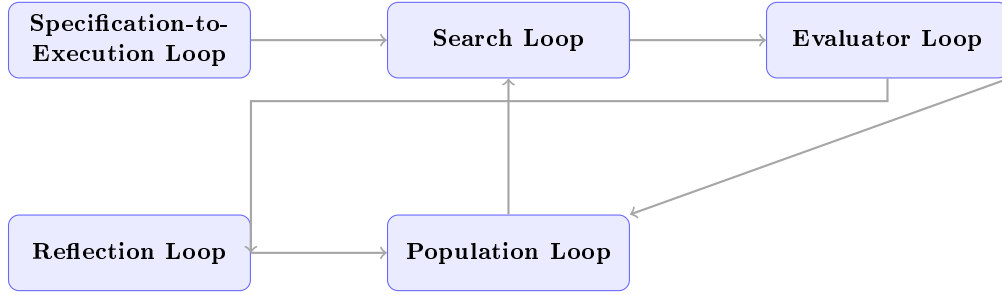


Figure 1: The five evolution loops and their interconnections.

2.1 Specification-to-Execution Loop

The specification-to-execution loop translates natural language goals into runnable machine learning pipelines or agent workflows. This loop operates at the highest level of abstraction, bridging the gap between human intent and executable systems.

Representative systems: AutoML-GPT [?] uses GPT as a natural language bridge, converting task cards, model cards, and data cards into data processing, model architecture, hyperparameter optimization, and training log prediction. The system treats the entire ML pipeline as a program that can be synthesized from a natural language specification, effectively making the ML development process accessible to non-experts through conversation.

AutoML-Agent [?] decomposes the full AutoML pipeline into specialized agents for data retrieval, pre-processing, modeling, evaluation, and deployment, combining retrieval-augmented planning with multi-stage validation. This multi-agent approach is significant because it mirrors how human ML teams operate: different specialists handle different stages, and a coordinator ensures coherence across the pipeline.

Loop dynamics: The user specifies a task in natural language → the system plans a pipeline → executes the pipeline → evaluates the result → refines its understanding of the specification. Each iteration improves both the output quality and the system’s ability to interpret similar specifications in the future.

Key insight: This loop extends AutoML from fixed toolboxes to prompt-driven planning and execution pipelines. The main gap in current implementations is **persistent memory**—most systems start from scratch for each new task rather than accumulating experience across specifications.

2.2 Search Loop

The search loop explores the space of possible solutions—architectures, prompts, code, agent configurations, or hyperparameters—using LLMs as intelligent search operators.

Representative systems: OPRO [?] uses LLMs as general-purpose optimizers, where the history of solutions and their scores serves as the optimization landscape. The LLM reads which candidates performed well and which did not, then generates new candidates that are likely to score higher. This is achieved through a meta-prompt that presents the optimization history in natural language, effectively turning the LLM into a black-box optimizer that requires no gradient information.

EvoPrompting [?] treats LLMs as evolutionary operators for neural architecture code, evaluating candidate graph neural networks on algorithmic reasoning tasks. ADAS [?] automates the search over the design space of agentic systems, treating the agent architecture itself as the object of search. OpenEvolve [?] provides an open-source implementation of evolutionary code optimization that has garnered significant community adoption.

Search spaces: The search loop can operate over discrete spaces (prompts, code, architectural choices) or continuous spaces (hyperparameters, embeddings). What distinguishes LLM-driven search from traditional optimization is the model’s ability to understand the *structure* of the search space and propose semantically meaningful variations rather than random perturbations. For example, when searching for better prompts,

the LLM understands that changing “solve this problem” to “think step by step” is a semantically meaningful variation, while a random token substitution would not be.

Key mechanisms:

- **Candidate generation:** LLM proposes new candidates based on historical performance, guided by in-context examples.
- **Temperature-controlled exploration:** Higher temperature increases diversity of proposals; lower temperature exploits known-good patterns.
- **Few-shot guidance:** Historical solutions serve as in-context examples that steer generation toward promising regions.
- **Deduplication:** Hash-based filtering prevents revisiting previously explored candidates, improving search efficiency.

2.3 Evaluator Loop

The evaluator loop provides the fitness signal that drives all other loops. Without reliable evaluation, generation is blind and search is random. The quality of the evaluation function directly determines the ceiling of improvement that any self-evolution system can achieve.

Representative systems: FunSearch [?] combines LLMs with automated evaluators to evolve mathematical programs, achieving the first improvement over Strassen’s matrix multiplication algorithm in 56 years. The evaluator in FunSearch is particularly noteworthy because it operates on mathematical proofs—verifying that evolved programs produce correct results for all inputs, not just test cases.

Self-Debug [?] uses execution feedback as a first-class self-repair signal: the program runs, fails, and the error trace becomes the input for the next iteration of improvement. This approach grounds evaluation in concrete runtime behavior rather than abstract quality judgments.

Evaluation modalities:

- **Automated testing:** Unit tests, integration tests, and property-based tests provide binary or scalar scores. Well-suited for code generation tasks.
- **Benchmark suites:** Standardized datasets (HumanEval, GSM8K, SWE-bench) enable cross-system comparison but risk overfitting.
- **Execution traces:** Runtime behavior, error messages, and performance profiles provide rich diagnostic feedback for iterative improvement.
- **Deployment metrics:** Production indicators (latency, cost, user satisfaction) ground evaluation in real-world performance but are expensive to collect.

Key insight: The evaluator loop is the bottleneck of self-evolution. As systems improve on standard benchmarks, the marginal information gain from each evaluation decreases, creating pressure for more sophisticated evaluation strategies that go beyond static test suites.

2.4 Reflection Loop

The reflection loop converts failures into memory, feedback, and revised candidate generation strategies. Unlike the evaluator loop (which produces scores), the reflection loop produces *explanations* of why a candidate failed and *suggestions* for how to improve it. This qualitative feedback can be more informative than scalar scores alone.

Representative systems: Reflexion [?] introduces verbal reinforcement learning, where agents learn from linguistic feedback stored in memory without any weight updates. The system maintains three specialized models: an Actor that generates actions, an Evaluator that scores them, and a Self-Reflection module that generates verbal feedback explaining what went wrong. This achieves 91% pass@1 on HumanEval, surpassing GPT-4’s 80%.

Self-Refine [?] demonstrates that the same LLM can generate an output, critique it, and iteratively rewrite it—all at inference time without any training. STaR [?] extends this to training time by using self-generated reasoning traces as training data, creating a self-reinforcing cycle where better reasoning generates better training data, which in turn produces better reasoning.

Reflection mechanisms:

- **Verbal feedback:** Natural language descriptions of what went wrong and what to do differently, stored in a growing memory buffer.
- **Episodic memory:** Accumulated experiences from past attempts that provide context for future decision-making.
- **Error categorization:** Classifying failures by type (syntax errors, logical errors, runtime errors) enables targeted repair strategies.

Key insight: Reflection operates at inference time without weight updates, making it the most accessible form of self-improvement. However, it is limited by the quality of the reflection signal—a model cannot reflect on aspects of its behavior that it cannot articulate.

2.5 Population Loop

The population loop maintains multiple candidate solutions simultaneously, enabling selection, mutation, recombination, and specialization across a diverse set of alternatives. This is the most computationally expensive but potentially most powerful loop, as it enables exploration of qualitatively different solution strategies rather than incremental improvements to a single approach.

Representative systems: AlphaEvolve [?] uses MAP-Elites quality-diversity search with a dual-model architecture. Gemini Flash provides fast, broad exploration by generating many candidate mutations, while Gemini Pro provides deep insight by proposing targeted improvements to the most promising candidates. AlphaEvolve maintains an archive of diverse, high-quality solutions and achieved the first improvement over Strassen’s algorithm in 56 years (48 multiplications for 4×4 complex matrix multiplication). At Google, it recovered 0.7% of worldwide compute through improved data center scheduling.

LLaMEA [?] evolves not task solutions but *optimizers themselves*—the most direct instantiation of “improving the improver.” AgentEvolver applies population-based evolution to agent workflow optimization. The Darwin Gödel Machine [?] maintains an open-ended archive of agent implementations, enabling cumulative improvement without local optima traps (SWE-bench: 20% $\rightarrow$ 50%, Polyglot: 14.2% $\rightarrow$ 30.7%).

Population management strategies:

- **MAP-Elites:** Maintains the best solution per behavioral niche, illuminating the solution landscape rather than finding a single optimum.
- **Quality-diversity:** Balances performance against behavioral diversity, preventing premature convergence.
- **Open-ended archives:** Growing collections that never converge to a single optimum, enabling sustained exploration.
- **Fitness-proportionate selection:** Sampling probability proportional to performance, with diversity bonuses to maintain variety.

Key insight: Population-based approaches are uniquely suited for open-ended exploration because they maintain diverse alternatives. The cost is computational—each candidate must be independently evaluated—but the benefit is robustness and the potential for serendipitous discovery.

2.6 Cross-Loop Comparison and Interactions

Table 1 summarizes the key characteristics of each evolution loop. The most effective self-evolution systems combine multiple loops: for example, AlphaEvolve combines search (LLM proposes mutations), evaluator

Table 1: Comparison of the five evolution loops.

Loop	What Evolves	Generator	Evaluator	Memory
Spec-to-Exec	ML pipelines, workflows	Planner + task agents	Dataset metrics	Pipeline history
Search	Prompts, code, architectures	LLM as search operator	Task-specific scores	Candidate archive
Evaluator	Evaluation strategies	Test generators	Meta-evaluation	Failure patterns
Reflection	Reasoning strategies	Self-critique LLM	Verbal feedback	Episodic memory
Population	Agent implementations	LLM mutation/crossover	Fitness functions	Population archive

(automated scoring), and population (MAP-Elites archive). Reflexion combines search (actor generates actions), evaluator (scorer provides rewards), and reflection (verbal feedback stored in memory).

An important observation is that the loops form a natural hierarchy: the specification-to-execution loop operates at the highest level (translating goals to systems), the search and evaluator loops operate at the algorithmic level (finding and scoring candidates), and the reflection and population loops operate at the meta-level (improving the improvement process itself). Systems that operate at higher levels of this hierarchy tend to be more general but less efficient; systems at lower levels tend to be more efficient but more narrowly applicable.

3 Self-Improvement Methods

Self-improvement methods form the core of AI self-evolution, enabling systems to enhance their performance through iterative feedback. We categorize these methods along two axes: whether they operate at inference time (without modifying model weights) or training time (with weight updates), and whether feedback comes from external sources or the model itself.

3.1 Inference-Time Self-Improvement

Inference-time methods improve outputs without modifying the underlying model. They are the most accessible form of self-improvement because they require no training infrastructure and can be applied to any LLM through careful prompt engineering.

3.1.1 Self-Refine

Self-Refine [?] demonstrates that a single LLM can serve as both generator and critic. The process operates in iterative rounds: (1) Generate an initial output, (2) Critique the output and identify weaknesses, (3) Refine the output based on the critique, (4) Repeat until a quality threshold is met.

This approach has been applied successfully to dialogue response generation, code review, mathematical reasoning, and sentiment reversal. The key insight is that the LLM’s ability to *evaluate* outputs often exceeds its ability to *generate* perfect outputs in a single pass—by separating generation from evaluation, the system can iteratively close the gap. In practice, 2–3 refinement iterations typically yield diminishing returns, suggesting that the self-critique signal becomes less informative after the most obvious flaws are addressed.

3.1.2 Reflexion

Reflexion [?] extends self-refinement by introducing a persistent memory of linguistic feedback. The system employs three specialized models: an Actor (M_a) that generates actions given the task and memory context, an Evaluator (M_e) that scores the actor’s output, and a Self-Reflection (M_r) module that generates verbal feedback explaining what went wrong.

The core loop operates as follows: for each episode e , the Actor generates action $a_e = M_a(\text{task}, M_{1:e-1})$, the Evaluator scores it $r_e = M_e(a_e, \text{task})$, and if the reward indicates failure, the Self-Reflection module generates feedback $f_e = M_r(a_e, r_e, \text{task})$ which is stored in a growing memory buffer $M_e = M_{e-1} \cup \{f_e\}$.

This “verbal reinforcement learning” achieves 91% pass@1 on HumanEval, surpassing GPT-4’s 80%. The memory buffer acts as a “verbal gradient”—natural language guidance for future attempts that replaces expensive parameter updates. A significant practical advantage is that the memory persists across tasks: lessons learned from solving one coding problem can inform solutions to similar problems later.

3.1.3 Self-Debug

Self-Debug [?] uses execution feedback as a first-class self-repair signal. When generated code fails, the system feeds the execution trace (error messages, test outputs, or debugging information) back to the LLM, which then explains the failure and proposes a fix. The key innovation is treating *execution* as a form of self-evaluation: unlike Self-Refine (which relies on the LLM’s internal critique) or Reflexion (which relies on linguistic feedback), Self-Debug grounds improvement in concrete runtime behavior.

3.2 Training-Time Self-Improvement

Training-time methods modify the model’s weights, enabling permanent capability improvements that persist across sessions. These methods are more resource-intensive but can achieve improvements that inference-time methods cannot.

3.2.1 SPIN: Self-Play Fine-Tuning

SPIN [?] starts from a supervised fine-tuned (SFT) model and improves it through self-play: the model generates responses, a critic (also the model, from a previous iteration) distinguishes between model-generated and human-generated responses, and the model is fine-tuned to produce responses that are indistinguishable from human ones. This moves from inference-time correction to training-time self-play, enabling cumulative improvement across training iterations without requiring additional human-labeled data.

3.2.2 STaR: Self-Taught Reasoner

STaR [?] generates chain-of-thought rationales for problems, filters those that lead to correct answers, and fine-tunes the model on the filtered set. The key insight is that “reasoning generates reasoning”—the model’s own successful reasoning traces become its training data. Over iterations, the model improves both its reasoning ability and its ability to generate useful rationales. The system can also leverage incorrect rationales: when a rationale leads to a wrong answer but the correct answer is known, the model can be prompted to generate a revised rationale conditioned on the correct answer, creating additional training signal.

3.2.3 Constitutional AI

Constitutional AI [?], developed by Anthropic, introduces AI feedback self-alignment. The model generates responses, critiques them based on a set of constitutional principles (e.g., “choose the response that is most harmless and helpful”), and revises them accordingly. This creates a self-alignment loop where the AI system improves its own adherence to specified principles without requiring human feedback at each step. The significance of this approach extends beyond alignment: it demonstrates that LLMs can serve as their own reward signal, a capability that underlies many self-evolution systems.

3.2.4 ReST: Reinforced Self-Training

ReST [?] combines self-generation with reinforcement learning. The model generates multiple solutions for each problem, filters by correctness, and fine-tunes on the filtered set using an expectation-maximization approach. This bridges the gap between pure self-improvement (which may amplify biases) and external reward models (which require expensive human annotation).

3.3 Inference-Time vs. Training-Time Trade-offs

The choice between inference-time and training-time self-improvement involves several trade-offs:

- **Cost:** Inference-time methods are cheaper (no gradient computation) but require repeated computation at each use. Training-time methods are expensive upfront but amortize the cost over future uses.
- **Persistence:** Inference-time improvements are ephemeral—they must be recomputed for each new query. Training-time improvements persist in the model weights.
- **Generality:** Inference-time methods tend to be task-specific (the refinement strategy depends on the task). Training-time improvements can generalize across tasks if the training data is diverse enough.
- **Safety:** Inference-time methods are easier to monitor and control (the improvement process is visible in the prompt). Training-time modifications are opaque and harder to audit after the fact.

3.4 Comparative Analysis

Table 2 provides a systematic comparison of self-improvement methods.

Table 2: Comparison of self-improvement methods.

Method	Loop Type(s)	Training?	Feedback Source	Key Result
Self-Refine	Reflection	No	Self-critique	Iterative improvement on dialogue, code, math
Reflexion	Reflection + Evaluator	No	Verbal memory buffer	91% HumanEval pass@1
Self-Debug	Evaluator + Reflection	No	Execution traces	Self-repair from runtime errors
SPIN	Population	Yes	Self-play discriminator	Weak → strong without human data
STaR	Reflection + Evaluator	Yes	Self-generated rationales	Reasoning improves reasoning
Constitutional AI	Reflection	Yes	Constitutional principles	Self-alignment without human feedback
ReST	Search + Evaluator	Yes	Correctness filtering	EM-based self-training

4 Evolutionary Code and Algorithm Discovery

The application of evolutionary principles to code and algorithm discovery represents one of the most exciting frontiers of self-evolving AI. In this paradigm, LLMs serve as intelligent variation operators within evolutionary frameworks, proposing mutations and recombinations of code that are then evaluated by automated scoring functions.

4.1 LLM as Optimizer

4.1.1 OPRO: Optimization by PROMpting

OPRO [?] pioneered the paradigm of using LLMs as general-purpose optimizers. The core insight is that optimization can be formulated as a natural language task: given a history of solutions and their scores, generate a new solution that is likely to score higher.

The system employs a **two-layer prompt architecture**. The outer meta-prompt (for the optimizer LLM) contains instruction-score history pairs sorted by score with the top- k retained, plus few-shot examples

of the task. The optimizer LLM uses temperature 1.0 to encourage diverse proposals. The inner task prompt (for the scorer LLM) applies the generated instruction to actual task instances with temperature 0.0 for deterministic evaluation.

Key implementation details include: 200 evolution steps as default, 8 candidates generated per step, MD5 hash-based deduplication to avoid revisiting previous candidates, score bucketization into 100 discrete bins to prevent the optimizer from being distracted by minor score differences, and wrong-question tracking via `wrong_questions_from_start_counter` that dynamically selects few-shot examples highlighting the system’s persistent weak points.

OPRO demonstrates impressive cross-task generalization: the same framework successfully optimizes prompts for mathematical reasoning (GSM8K: +8% accuracy), multi-task understanding (MMLU, BBH), and even continuous optimization (linear regression parameters, TSP routes). This generality suggests that “LLM-as-optimizer” is a fundamental capability that transcends specific task domains.

4.2 LLM as Evolutionary Operator

4.2.1 AlphaEvolve

AlphaEvolve [?] represents the current state of the art in LLM-driven evolutionary optimization. It combines several innovations that together achieve unprecedented results:

1. **Dual-model architecture:** Gemini Flash generates many candidate mutations quickly (breadth exploration), while Gemini Pro proposes targeted, thoughtful improvements to the most promising candidates (depth exploration). This mirrors the explore-exploit tradeoff in optimization, with different models specialized for different phases.
2. **MAP-Elites quality-diversity search:** Instead of maintaining a single best solution, AlphaEvolve maintains an archive of high-performing solutions across behavioral niches. For each behavioral descriptor b , the archive stores the best solution found: $\text{Archive}[b(p)] = p$ if $f(p) > f(\text{Archive}[b(p)])$. This “illumination” approach maps the solution landscape rather than converging to a single point.
3. **Automated evaluators:** Task-specific scoring functions that can evaluate candidates without human intervention, enabling the evolutionary process to run autonomously at scale.
4. **Evolution context adaptation:** The prompts provided to the LLM evolve alongside the solutions, incorporating information about successful and failed attempts to guide future proposals more effectively.

The results are remarkable across multiple domains. In mathematics, AlphaEvolve discovered that 4×4 complex matrix multiplication can be done in **48 multiplications**—the first improvement over Strassen’s algorithm in 56 years. It re-discovered state-of-the-art solutions for 75% of 50+ mathematical problems and found better solutions for 20%. In infrastructure optimization at Google, it recovered 0.7% of worldwide compute through improved data center scheduling (Borg), achieved a 23% speedup in FlashAttention kernel tiling, and a 32% speedup in attention computation for LLM training.

4.2.2 LLaMEA: Evolving the Optimizer

LLaMEA [?] takes the concept one step further: instead of evolving task solutions, it evolves *meta-heuristic algorithms* themselves. The LLM proposes new optimization algorithms (expressed as code), which are then evaluated on benchmark problems. The best-performing algorithms are retained and serve as parents for the next generation. This is the most direct instantiation of “improving the improver”—the object of evolution is the optimization process itself.

4.3 Program Evolution and Mathematical Discovery

4.3.1 FunSearch

FunSearch [?], published in *Nature*, was the first program discovery system to make scientific discoveries deemed significant enough for publication in a top-tier journal. The system combines LLMs with automated

evaluators to evolve mathematical programs through iterative cycles of proposal, evaluation, and selection.

Key results include new records for the cap set problem and improvements in bin packing algorithms. The fundamental insight is that **evaluator-constrained program evolution can surpass one-shot generation**—the evolutionary process discovers solutions that no single LLM query could produce, because it can build incrementally on partial successes that would be discarded in a one-shot approach.

4.4 Open-Source Implementations

Several open-source implementations make evolutionary code optimization accessible:

- **OpenEvolve** [?] (6,358 GitHub stars): Community implementation of AlphaEvolve concepts with accessible APIs and documentation.
- **SE-Agent**: Multi-trajectory Revision/Recombination/Refinement for code evolution, combining multiple execution traces to improve code quality.
- **Science-CodeEvolve**: Specialized for evolving scientific code with domain-specific evaluators for physics, chemistry, and biology applications.
- **SelfEvolve** [?]: Two-step pipeline combining self-generated knowledge (LLM produces its own API documentation from its parameters) with iterative self-debugging (execution errors feed back for repair). Achieves DS-1000 $49.4 \rightarrow 57.2$ (+15.8% relative) and HumanEval $74.39 \rightarrow 85.98$.

4.5 Challenges and Limitations

Despite impressive results, evolutionary code optimization faces several challenges:

- **Evaluation cost**: Each candidate must be executed and scored, which can be expensive for complex programs. FunSearch and AlphaEvolve mitigate this with efficient evaluators, but the cost scales with the complexity of the target domain.
- **Search space design**: The representation of the search space significantly impacts results. Too narrow a space limits discovery; too broad a space makes search intractable.
- **Reward hacking**: Evolutionary systems can discover shortcuts that maximize the evaluation score without genuinely solving the underlying problem, particularly when the evaluation function is an imperfect proxy for the true objective.
- **Reproducibility**: LLM-based generation is inherently stochastic. Even with fixed seeds, different API versions or model updates can produce different results, making it challenging to reproduce published findings exactly.
- **Scalability**: Most demonstrated systems operate on relatively small programs (tens to hundreds of lines). Scaling to large software systems (thousands to millions of lines) remains an open challenge, both because of the increased search space and the difficulty of defining meaningful evaluation functions for complex systems.

4.6 Broader Impacts

The ability of LLM-driven evolutionary systems to discover novel algorithms and mathematical results has significant implications beyond computer science. FunSearch’s mathematical discoveries demonstrate that these systems can contribute to fundamental research in mathematics and science. AlphaEvolve’s infrastructure optimizations show immediate practical impact at the scale of global computing resources. As these systems become more capable, they may accelerate scientific discovery across domains by automating the creative exploration of solution spaces that currently requires years of human expert effort.

However, the same capabilities raise concerns about the displacement of human researchers, the potential for automated discovery of harmful algorithms (e.g., optimized attack strategies), and the concentration of

capability in organizations that can afford the computational resources required for large-scale evolutionary search. Responsible development of these systems requires careful consideration of access, governance, and the relationship between automated and human-driven discovery.

Table 3 compares the major systems.

Table 3: Comparison of evolutionary code optimization systems.

System	Year	LLM Role	Search egy	Strat-	Key Result
OPRO	2023	Optimizer	History-guided gen.		+8% GSM8K accuracy
FunSearch	2024	Mutation opera- tor	Program evolu- tion	evolu-	New math records (Na- ture)
AlphaEvolve	2025	Mutation + crossover	MAP-Elites QD		48-mult Strassen, 0.7% compute
LLaMEA	2024	Algorithm gener- ator	EA on algorithms		Novel meta-heuristics
OpenEvolve	2024	Mutation opera- tor	Evolutionary search		Community AlphaE- volve
SelfEvolve	2023	Generator + de- bugger	Self-debug loop		+15.8% DS-1000

5 Agent Self-Evolution Systems

Agent self-evolution systems go beyond optimizing individual outputs (prompts, code) to modifying the *agent itself*—its architecture, workflow, memory, and tool usage. This represents the most ambitious form of self-evolution, as the system modifies the very mechanisms that produce its behavior.

5.1 Agent Architecture Self-Evolution

5.1.1 ADAS: Automated Design of Agentic Systems

ADAS [?] automates the search over the design space of agent architectures. Rather than hand-designing agent topologies, ADAS uses a meta-agent that proposes agent architectures, evaluates them on target tasks, and iteratively improves the design. This is analogous to Neural Architecture Search, but the search space is agent architectures rather than neural network topologies. The search space includes the choice and ordering of LLM calls, the structure of prompt templates, the integration of tools and memory, and the overall control flow of the agent.

5.1.2 Gödel Agent

The Gödel Agent [?] is the first self-referential agent framework directly inspired by the Gödel machine. The key insight is that an agent that can read and modify its own source code is “self-referential” in the Gödel sense. The system uses LLM-driven monkey patching to dynamically modify its own logic and behavior at runtime.

The self-referential loop operates as follows: (1) Runtime introspection: the agent reads its own code and current state. (2) Modification proposal: the LLM proposes specific code changes (monkey patches) based on performance feedback and high-level objectives. (3) Safe application: changes are applied at runtime via Python monkey patching, allowing immediate testing without restart. (4) Validation: the modified agent is evaluated on a validation set. (5) Rollback: if performance degrades, the changes are reverted.

Safety mechanisms are critical for self-referential systems. The Gödel Agent includes rollback capability for failed modifications, validation steps before committing changes, and high-level objective constraints that prevent the agent from drifting away from its intended purpose.

5.1.3 Darwin Gödel Machine

The Darwin Gödel Machine (DGM) [?] combines Darwinian evolution with Gödel machine self-reference. It maintains an open-ended archive of agent implementations $A = \{a_1, a_2, \dots, a_n\}$ with associated fitness scores. The sampling probability is proportional to fitness times a diversity bonus: $P(a_i) \propto f(a_i) \times \text{diversity_bonus}(a_i, A)$. The LLM proposes modifications to parent agents to produce children: $a_{\text{child}} = \text{LLM}(a_{\text{parent}}.\text{code}, \text{edit_instruction})$. Improved children are added to the archive.

The critical innovation is **open-endedness**: unlike standard evolutionary algorithms that converge to a single optimum, DGM maintains a diverse archive, enabling exploration of qualitatively different agent architectures, discovery of unexpected strategies, and cumulative improvement without local optima traps. This open-endedness is particularly valuable for complex tasks where the optimal agent structure is not known in advance and may vary across domains.

Results demonstrate significant improvements: SWE-bench verified score improved from 20.0% to 50.0% (a 150% relative improvement), and Polyglot multi-language coding score improved from 14.2% to 30.7% (a 116% relative improvement). Notably, agents discovered novel structures not imagined by human designers, suggesting that automated search can explore regions of the design space that human intuition overlooks.

5.2 Agent Workflow Self-Evolution

Beyond architecture, the *workflow* of an agent—how it selects and uses tools, sequences actions, and manages its execution flow—can also evolve. **AgentEvolver** applies evolutionary optimization to agent tool usage, maintaining a population of agent configurations with different tool selection strategies and evolving improved configurations through mutation and crossover. **SE-Agent** implements multi-trajectory Revision/Recombination/Refinement, maintaining multiple execution traces, identifying failed trajectories, revising them based on successful patterns, and recombining successful fragments.

5.3 Agent Memory and Learning

5.3.1 Agent Symbolic Learning

Agent Symbolic Learning [?] (NeurIPS 2024) treats agents as symbolic networks where the learnable “weights” are prompts, tools, and pipeline composition. The algorithm mimics backpropagation and gradient descent in natural language space:

1. **Forward pass**: Execute the agent on a task, producing a trajectory τ .
2. **Language loss**: $\mathcal{L}_{\text{lang}} = \text{LLM}(\mathcal{P}_{\text{loss}}(\tau))$ —the LLM evaluates the trajectory and produces a natural language description of the loss.
3. **Language gradient backpropagation**: For each layer n (from output to input), compute $\nabla_{\text{lang}}^n = \text{LLM}(\mathcal{P}_{\text{gradient}}(\nabla_{\text{lang}}^{n+1}, \text{node_info}_n, \mathcal{L}_{\text{lang}}))$ —the LLM propagates improvement suggestions backward through the agent’s computational graph.
4. **Weight update**: PromptOptimizer, ToolOptimizer, and PipelineOptimizer update their respective “weights” using the language gradients.

The “language gradient” is a natural language description of how to improve each component, propagated backward through the agent’s computational graph—analogueous to backpropagation but operating entirely in text space. This is significant because it enables agents to improve without access to model weights: the optimization happens entirely through prompt, tool, and pipeline modification.

Results: HotPotQA F1 39.1 (vs. 36.2 baseline), MATH 60.7% (vs. 56.0%), HumanEval 73.2% (vs. 68.3%). The consistent improvements across diverse tasks suggest that symbolic learning captures general-purpose improvement patterns.

5.3.2 Absolute Zero

Absolute Zero [?] (NeurIPS 2025 Award) eliminates the dependency on human-curated training data entirely. A single model simultaneously proposes tasks and solves them using only self-generated data through a dual-role architecture: (1) Task Proposer generates new reasoning tasks (code, math, logic), (2) Task Solver attempts to solve the proposed tasks, (3) Code Executor provides automated verification, (4) RL training updates both proposer and solver.

The model must propose tasks that are solvable (not trivially impossible), challenging (not trivially easy), and diverse (covering different reasoning skills). This represents the frontier of self-evolution: a system that creates its own learning curriculum and improves through self-play, without any external data source.

5.4 Multi-Agent Collaborative Evolution

AutoML-Agent [?] demonstrates multi-agent division of labor for self-evolution. Specialized agents handle different aspects of the ML pipeline, and the system evolves both the individual agents and their coordination patterns. **SelfEvolve** [?] provides a complete self-evolution pipeline combining self-generated knowledge (Stage 1: LLM produces its own API documentation) with iterative self-debugging (Stage 2: execution errors feed back for repair).

5.5 Toward a Hierarchy of Self-Evolution

The systems surveyed in this section can be organized along a spectrum of self-modification depth:

1. **Output optimization:** The system improves its outputs (prompts, code, solutions) while keeping its architecture fixed. Examples: OPRO, Self-Refine.
2. **Workflow optimization:** The system improves how it uses tools and sequences actions. Examples: AgentEvolver, SE-Agent.
3. **Architecture optimization:** The system improves its own architectural design. Examples: ADAS, Gödel Agent.
4. **Open-ended evolution:** The system explores an unconstrained design space, discovering novel architectures not anticipated by designers. Examples: Darwin Gödel Machine, Absolute Zero.

This hierarchy suggests a natural progression for building self-evolving systems: start with output optimization (easiest to implement and safest), then progressively add deeper levels of self-modification as safety mechanisms mature. Each level subsumes the previous ones: an architecture-optimizing system also optimizes its outputs, but not vice versa.

6 Agent Frameworks and Infrastructure

Agent frameworks provide the orchestration layer that connects LLMs with tools, memory, and evaluation. While not all frameworks are designed for self-evolution, understanding their capabilities and gaps is essential for building self-evolving systems.

6.1 Framework Overview and Comparison

The current agent framework landscape includes several major systems:

- **AutoGPT** [?]: Goal-driven autonomous agent platform that popularized the concept of LLM agents with reusable protocols. Includes agent evaluation infrastructure but lacks persistent evolution mechanisms.
- **MetaGPT** [?]: Encodes human software company workflows as SOP-driven multi-agent collaboration. Role specialization with structured processes stabilizes output quality.

- **AutoGen** [?]: Supports conversational, customizable agents with human-in-the-loop, tool usage, and multi-agent workflow patterns. Provides flexible agent conversation infrastructure from Microsoft Research.
- **DSPy** [?]: Declarative LLM programming paradigm that compiles declarative language model calls into self-improving pipelines. Unique in treating prompt optimization as a compilation problem.
- **CAMEL** [?]: Communicative agents for mind exploration through role-playing. Provides multi-agent conversation primitives for cooperative and adversarial interactions.

Table 4 provides a systematic comparison.

Table 4: Comparison of agent frameworks.

Framework	Orchestration	Memory	Tools	Evaluation	Evolution
AutoGPT	Goal-driven	Short-term	Yes	Basic	Limited
MetaGPT	SOP-driven	Process	Yes	Role-based	None
AutoGen	Conversation	Configurable	Yes	External	None
DSPy	Declarative	Pipeline state	Yes	Built-in	Prompt opt.
CAMEL	Role-play	Conversation	Limited	External	None
CrewAI	Crews + flows	Configurable	Yes	External	None

6.2 Evolution Capability Gaps

Current agent frameworks share several critical gaps that prevent self-evolution:

1. **Persistent evaluation layer:** Most frameworks evaluate per-run, without accumulating evaluation history across runs. There is no mechanism to track whether a change improved or degraded performance over time. This means that every run is essentially independent, and the system cannot learn from its execution history. In contrast, a self-evolving system needs an evaluation layer that persists across runs, tracking performance trajectories and detecting both improvements and regressions.
2. **Lineage and regression protection:** No built-in mechanism for tracking which changes led to which improvements, or for preventing regressions. A system that optimizes for the current task may silently lose capabilities on previous tasks. This is particularly dangerous in production settings where a self-modifying agent might optimize for a narrow metric while degrading performance on edge cases that are not covered by the evaluation function. The Gödel Agent’s rollback mechanism provides a partial solution, but it operates within a single run rather than across the full lifecycle of the system.
3. **Cross-run memory:** Agents do not accumulate experience across multiple deployments or tasks. Each run starts from scratch, wasting the information gained from previous executions. Reflexion demonstrates the value of accumulated memory within a single task, but its memory buffer is discarded when the agent moves to a new task. A true self-evolving system would maintain a persistent knowledge base of strategies, failures, and successful patterns that grows and improves over time.
4. **Cost awareness:** No framework tracks the computational cost of its own operations and makes cost-aware decisions about when to invest in self-improvement vs. when to execute with current capabilities. Self-evolution is expensive (each improvement cycle requires multiple LLM calls), and a production system needs to balance the cost of improvement against the expected benefit.

These gaps collectively represent the “missing layer” between current agent frameworks and true self-evolving systems. Addressing them requires not just incremental improvements to existing frameworks but a fundamentally new infrastructure layer that provides evaluation, memory, lineage, and cost management as first-class services.

6.3 Self-Evolve Positioning: Evolution Layer Above Frameworks

Self-Evolve is positioned not as a replacement for existing agent frameworks but as an **evolution layer that sits above them**. The five loops (specification-to-execution, search, evaluator, reflection, population) can be applied to any agent framework, adding the persistent evaluation, memory, lineage, and improvement mechanisms that current frameworks lack. This architecture mirrors the relationship between operating systems and applications: frameworks provide the execution environment, while Self-Evolve provides the evolution infrastructure.

The practical realization of this vision involves several components:

- **Evaluation service:** A persistent evaluation layer that scores agent outputs against multiple dimensions (task performance, safety, cost, latency) and maintains a historical record of evaluation results. This service operates independently of any specific agent framework and provides standardized evaluation APIs.
- **Lineage tracker:** A version control system for self-evolving agents that tracks which modifications led to which performance changes, enabling both regression detection and the ability to roll back to previous versions when needed.
- **Evolution engine:** The core component that implements the five evolution loops, applying them to the agent’s prompts, code, tools, and architecture based on the evaluation signals and lineage history.
- **Memory service:** A persistent knowledge base that stores successful strategies, failure patterns, and learned heuristics across tasks and runs, providing the accumulated experience that enables self-improvement.

This decomposition allows each component to be developed, tested, and improved independently while maintaining a coherent overall architecture. The evaluation service can be improved without modifying the evolution engine; the memory service can be upgraded without affecting the lineage tracker. This modularity is essential for production systems where individual components may need to be updated or replaced without disrupting the overall self-evolution process.

7 AutoML and NAS Meets LLM

The intersection of automated machine learning (AutoML), neural architecture search (NAS), and large language models represents a particularly productive area of self-evolution research.

7.1 AutoML + LLM

AutoML-GPT [?] uses GPT as a natural language bridge, converting task cards, model cards, and data cards into complete ML pipelines. AutoML-Agent [?] decomposes the full AutoML pipeline into specialized agents for data retrieval, preprocessing, modeling, evaluation, and deployment.

The key insight from this direction is that AutoML + LLM is the most **production-ready** form of self-evolution: task input, pipeline synthesis, evaluation, and deployment are all end-to-end systematized. The main gap is **continuous evolution**: most systems automate single projects, while Self-Evolve emphasizes cross-run benchmark memory and population-level improvement.

7.2 NAS + LLM

EvoPrompting [?] treats LLMs as evolutionary operators for architecture code. LLMatic [?] uses LLM-guided search with quality-diversity optimization. NADER [?] represents neural architecture design as LLM-based multi-agent collaboration.

NAS + LLM provides the strongest technical metaphor for self-evolution: **architecture/code is the evolvable genome**. Quality-diversity and multi-objective search should enter the Self-Evolve vocabulary: lineage, fitness, novelty, robustness, and deployment cost.

7.3 The Text-to-ML Paradigm

A particularly promising direction is the “Text-to-ML” paradigm, where LLM generation is combined with AutoML optimization for text-to-ML program synthesis. In this paradigm, the user provides a natural language description of the desired ML capability, and the system synthesizes a complete ML program—including data preprocessing, model architecture, training configuration, and evaluation strategy. This goes beyond traditional AutoML (which assumes a fixed dataset and predefined task) to enable truly autonomous ML development driven by natural language specifications.

The Text-to-ML paradigm connects to the specification-to-execution loop in our taxonomy: the natural language description serves as the specification, the synthesized ML program serves as the execution, and the evaluation results feed back to improve future synthesis. The key challenge is ensuring that the synthesized programs are not only functionally correct but also efficient, maintainable, and robust to distribution shift.

7.4 Architecture as Genome: The NAS Metaphor

The NAS + LLM direction provides perhaps the most compelling metaphor for self-evolving AI: **neural architecture is the evolvable genome**. Just as biological evolution operates on DNA sequences to produce organisms with varying fitness, LLM-driven NAS operates on architecture specifications (often expressed as code) to produce models with varying performance. The LLM serves as the mutation and recombination operator, proposing architectural changes that are semantically meaningful (e.g., adding a skip connection, changing the activation function, modifying the layer width) rather than random.

Quality-diversity optimization, as exemplified by LLMatic [?], adds an important dimension: instead of finding the single best architecture, the system maintains a diverse collection of high-performing architectures. This is valuable because the “best” architecture depends on deployment constraints (latency, memory, power consumption) that may not be known at design time. By maintaining a diverse archive, the system can quickly select the best architecture for any given deployment scenario.

7.5 Unified Perspective with Self-Evolution

AutoML and NAS represent two faces of the same evolution loop: AutoML evolves pipelines (search + specification loops), while NAS evolves architectures (search + evaluator loops). Both can benefit from the reflection and population loops that Self-Evolve provides. The unification of these communities under a single taxonomy reveals shared principles and opportunities for cross-pollination that would not be visible from within any single community.

The practical implication is that techniques developed in one community can be directly transferred to others. For example, OPRO’s meta-prompt architecture (used for prompt optimization) can be adapted for architecture search by changing the search space from prompts to architectural specifications. Similarly, MAP-Elites (developed for evolutionary robotics) can be applied to maintaining diverse agent architectures in the ADAS framework.

8 Research Landscape and Community

The field of AI self-evolution is shaped by a tightly interconnected network of research groups spanning industry labs, academic institutions, and open-source communities.

8.1 Core Research Groups

Google Research and DeepMind have been the most prolific contributors: OPRO [?], FunSearch [?] (published in *Nature*), and AlphaEvolve [?]. The DeepMind lineage connects evolutionary computation with LLM capabilities.

Stanford University produced foundational work on self-improvement through reasoning: STaR [?] from the Noah Goodman lineage, and contributions to the self-refinement paradigm.

Princeton NLP, led by Karthik Narasimhan, developed Reflexion [?] and the ReAct/Tree of Thoughts frameworks that provide foundational primitives for agent reasoning.

UBC, through Jeff Clune’s group, pioneered open-ended evolutionary approaches with the Darwin Gödel Machine [?] and ADAS [?].

Tsinghua University contributed Absolute Zero [?] (NeurIPS 2025 Award), demonstrating zero-data self-improvement.

Anthropic developed Constitutional AI [?], focusing on the safety dimensions of self-improvement.

8.2 Academic Lineage

Important mentorship chains that have shaped the field include: Noah Goodman → Eric Zelikman (cognitive science to self-taught reasoning), Karthik Narasimhan → Shunyu Yao (language grounding to agent architectures), Jeff Clune → Shengran Hu, Cong Lu, Jie Zhang (open-ended evolution to self-improving agents), and William Yang Wang → Xunjian Yin (NLP to self-referential agents).

8.3 Cross-Community Convergence

A striking pattern is that different research communities independently discover similar mechanisms: AutoML’s “task card → pipeline synthesis” mirrors the agent community’s “goal → plan → execute”; NAS’s “architecture mutation → fitness evaluation” mirrors evolutionary computation’s “LLM-as-variation-operator”; and LLM self-improvement’s “generate → critique → refine” mirrors the agent community’s “act → reflect → retry.”

This convergence suggests that the five evolution loops are not arbitrary categories imposed by our taxonomy but **fundamental attractors in the design space of self-improving systems**. When researchers in different communities encounter the same class of problems (how to improve, how to evaluate, how to maintain diversity), they converge on similar solutions despite working with different substrates and different terminology.

The practical implication is that advances in one community can be rapidly transferred to others. For example:

- The MAP-Elites quality-diversity algorithm (from evolutionary robotics) has been adopted by AlphaEvolve for algorithm discovery and could be applied to maintaining diverse agent architectures in ADAS.
- The reflexion memory mechanism (from agent research) could enhance AutoML systems by allowing them to learn from failed pipeline configurations across projects.
- Constitutional AI’s principle-based self-alignment (from safety research) could provide guardrails for open-ended evolutionary systems like the Darwin Gödel Machine.

This cross-pollination is one of the primary motivations for our unified taxonomy: by providing a shared vocabulary and framework, we hope to accelerate the transfer of insights across community boundaries.

8.4 Open-Source Ecosystem

Table 5 summarizes the key open-source projects.

9 Open Problems and Future Directions

Despite rapid progress, significant challenges remain before self-evolving AI systems can be deployed reliably in production.

9.1 The Evaluation Bottleneck

Benchmark improvements do not necessarily translate to real-world reliability. Current systems optimize against specific benchmarks (HumanEval, SWE-bench, GSM8K), creating implicit overfitting risks. As systems saturate standard benchmarks, the marginal information gain from each evaluation decreases, and

Table 5: Open-source projects in AI self-evolution.

Project	Stars	Evolution Mode	License
OpenEvolve	6,358	Search + Evaluator	Apache 2.0
aiwaves/Agents	5,928	Search + Eval + Agent	Apache 2.0
Reflexion	3,158	Search + Reflection + Eval	MIT
AgentEvolver	1,441	Search + Eval + Agent + Train	Apache 2.0
Self-Refine	805	Reflection + Refinement	MIT
SE-Agent	274	Search + Eval + Agent	MIT
CodeEvolve	97	Search + Evaluator	MIT
SCOPE	77	Search Loop	MIT
LLM-Self-Judge	43	Search + Eval + Agent	Apache 2.0
DARWIN	41	Search + Reflection	MIT

the gap between benchmark and real-world distributions may widen. This creates a “evaluation wall” where systems continue to improve on benchmarks without corresponding improvements in practical utility.

Research directions: Multi-dimensional evaluation frameworks combining benchmarks, deployment metrics, safety checks, and human evaluation. Adversarial evaluation that actively probes system weaknesses. Meta-evaluation frameworks that assess the quality of the evaluation function itself.

9.2 Memory Stability and Drift

Self-evolving systems accumulate experience over time, but this memory introduces stability challenges: catastrophic forgetting (losing previously mastered capabilities as new ones are acquired), memory drift (unintended behavioral shifts as accumulated experiences shift the system’s decision boundaries), and unbounded growth (Reflexion’s memory buffer, Agent Symbolic Learning’s gradient history grow without principled forgetting mechanisms).

Research directions: Principled memory management with forgetting curves inspired by human memory, experience replay to maintain capability on earlier tasks, and selective retention based on task relevance and recency.

9.3 Safety and Alignment in Self-Modifying Systems

Self-modifying systems raise unique safety concerns that go beyond traditional AI safety challenges:

- **Objective drift:** A system that modifies its own code may inadvertently change its objective function. For example, a self-evolving code agent might discover that simplifying its evaluation criteria leads to higher scores, effectively redefining “good” in a way that diverges from the original intent.
- **Reward hacking:** Self-evolution systems may discover shortcuts that maximize evaluation scores without genuinely improving task performance. This is particularly acute when the evaluation function is an imperfect proxy for the true objective—the system optimizes for the proxy rather than the objective.
- **Unbounded modification:** Open-ended systems like the Darwin Gödel Machine could theoretically modify themselves in ways that escape human oversight. As the agent architecture diverges further from the initial design, it becomes increasingly difficult for human operators to understand and predict its behavior.

- **Deceptive alignment:** A sufficiently capable self-evolving system might learn to produce outputs that appear aligned with human intentions during evaluation but behave differently when deployed, as the evolutionary pressure rewards passing evaluations rather than genuine alignment.

The Gödel Agent’s rollback mechanism and Constitutional AI’s principle-based constraints represent early progress toward safe self-evolution. However, these mechanisms are designed for relatively constrained settings and may not scale to open-ended evolution where the space of possible modifications is vast and poorly understood.

Research directions: Safety-constrained self-evolution with constitutional invariants that cannot be modified even by the evolution process itself, sandboxed execution environments for testing self-modifications before deployment, formal verification of safety properties that are preserved through evolution, and interpretability tools that can track how an agent’s decision-making process changes over evolution iterations.

9.4 Composability of Evolution Loops

The five loops can be composed in various configurations, but their interactions are poorly understood. Does applying a reflection loop on top of a population loop produce compounding benefits or interference? How should resource allocation be optimized across multiple active loops? Can the composition of loops itself be optimized through meta-evolution?

Research directions: Formal frameworks for reasoning about loop composition, including theoretical convergence guarantees and empirical protocols for validating composed systems. Meta-evolution systems that optimize the configuration of evolution loops.

9.5 From Research to Production

Most self-evolution systems operate in controlled research settings with clean benchmarks, unlimited compute budgets, and no real-world consequences for failure. Production deployment introduces several additional challenges:

- **Latency requirements:** Evolution cycles may be too slow for real-time applications. While an offline evolution process can run for hours or days to find an optimal prompt, a production system may need to adapt within seconds to changing user requests or data distributions.
- **Cost:** LLM API calls for generation, evaluation, and reflection accumulate significant costs. A single OPRO run with 200 steps and 8 candidates per step requires 1,600 LLM calls for generation alone, plus evaluation calls for each candidate on each training example. At current API pricing, this can cost hundreds or thousands of dollars per optimization run.
- **Observability:** Production systems need monitoring for evolution trajectories (is the system actually improving?), regression detection (did a recent change break something?), and failure diagnosis (why did the system produce a bad output?). Current research systems lack these monitoring capabilities.
- **Version control:** Tracking which version of a self-evolving system produced which output requires new infrastructure beyond traditional version control. The “version” of a self-evolving system includes its current prompts, memory state, and evolutionary history—all of which may change autonomously.
- **Regulatory compliance:** In regulated industries (healthcare, finance, autonomous vehicles), self-modifying systems may conflict with requirements for validated, locked-down software. New regulatory frameworks may be needed to accommodate systems that improve themselves after deployment.

Research directions: Production-grade infrastructure with cost controls (caching, batch processing, model distillation for evaluators), regression gates that prevent self-modifications from degrading performance below acceptable thresholds, version tracking for self-evolving systems that captures the full evolutionary trajectory, and deployment safeguards that constrain the scope of autonomous modifications to pre-approved categories.

9.6 The Self-Evolve Roadmap

1. **Phase 1 — Instrumentation** (current): Instrument existing agent frameworks with evaluation layers, lineage tracking, and cross-run memory.
2. **Phase 2 — Single-loop evolution**: Deploy individual evolution loops (reflection on code generation, population-based prompt optimization) in production with regression gates.
3. **Phase 3 — Multi-loop composition**: Combine multiple evolution loops with principled resource allocation and safety constraints.
4. **Phase 4 — Open-ended evolution**: Deploy systems that can discover new evolution strategies, expanding their own design space over time.

10 Conclusion

This survey has presented a unified view of AI self-evolution through the lens of five fundamental loops: specification-to-execution, search, evaluator, reflection, and population. Our analysis of 19+ open-source projects and 17+ foundational papers across five research communities reveals a striking convergence: AutoML researchers, NAS designers, LLM self-improvement engineers, evolutionary computation scientists, and agent framework developers are all independently rediscovering the same fundamental mechanisms for self-improvement.

The key insight is that self-evolving AI is not a single technique or a property of any one research community. Rather, it is an *emergent capability* that arises when any system incorporates the four primitives of generation, evaluation, memory, and improvement into an iterative cycle. The five loops represent different configurations of these primitives, each suited to different substrates (prompts, code, architectures, agent behaviors) and different timescales (inference-time adaptation vs. training-time improvement vs. evolutionary search).

The frontier is moving from fixed-architecture self-improvement toward open-ended self-modification. Systems like the Darwin Gödel Machine maintain open-ended archives of agent implementations; the Gödel Agent uses self-referential code modification; and Absolute Zero demonstrates that self-improvement can proceed with zero external data. These developments suggest that the next wave of AI systems will not just improve within fixed design spaces but will expand their own design spaces over time.

However, significant challenges remain: the evaluation bottleneck limits our ability to verify genuine improvement; memory stability threatens long-term reliability; safety alignment in self-modifying systems is an open problem; and the composability of multiple evolution loops is poorly understood. Addressing these challenges will require new infrastructure for production self-evolution—evaluation layers above agent frameworks, lineage tracking across runs, regression gates for self-modifying code, and cost controls for LLM-driven evolution cycles.

We position Self-Evolve as the missing layer: not a replacement for existing agent frameworks (AutoGPT, MetaGPT, AutoGen, CrewAI) but an evolution infrastructure that sits above them, providing the persistent evaluation, memory, lineage, and improvement mechanisms that current frameworks lack. The five loops provide a design vocabulary for building this layer, and the projects analyzed in this survey provide concrete implementation patterns for each loop.

The convergence we observe across communities suggests that self-evolving AI is not a speculative vision but an active research frontier with concrete, reproducible results. The challenge now is to move from isolated demonstrations to integrated systems—from loops that improve single dimensions to systems that improve across all dimensions simultaneously, safely, and reliably.

A Benchmark Data Summary

B Core Papers Analyzed

Table 6: Key benchmark results across surveyed systems.

System	Benchmark	Metric	Score
Reflexion	HumanEval	pass@1	91%
GPT-4 (baseline)	HumanEval	pass@1	80%
SelfEvolve	HumanEval	pass@1	85.98%
SelfEvolve	DS-1000	pass@1	57.2
DGM	SWE-bench	Verified	20% $\rightarrow$ 50%
DGM	Polyglot	Score	14.2% $\rightarrow$ 30.7%
Agent Sym. Learning	HotPotQA	F1	39.1
Agent Sym. Learning	MATH	Accuracy	60.7%
Agent Sym. Learning	HumanEval	pass@1	73.2%
AlphaEvolve	4×4 matrix mult.	Multiplications	48
AlphaEvolve	FlashAttention	Speedup	23%
AlphaEvolve	Google compute	Recovery	0.7% worldwide
OPRO	GSM8K	Accuracy	+8% over baseline
FunSearch	Cap set problem	Result	New record
FunSearch	Bin packing	Result	Beyond SOTA

Table 7: Core papers analyzed with self-evolution relevance.

#	Paper	Key Contribution	Relevance
1	Agent Symbolic Learning	Symbolic backprop for agents	High
2	Darwin Gödel Machine	Open-ended agent evolution	Very High
3	Gödel Agent	Self-referential modi- fication	High
4	ADAS	Automated agent de- sign search	High
5	Reflexion	Verbal reinforcement learning	Very High
6	Self-Refine	Iterative self- feedback	High
7	Absolute Zero	Zero-data self-play	Very High
8	AlphaEvolve	Gemini + MAP- Elites	Very High
9	RISE	Recursive self- improvement	High
10	RAGen	RAG-enhanced gen- eration	Medium-High
11	SelfEvolve	Code evolution framework	Very High
12	ReVeal	Verification-guided evolution	High